データ鮮度・学習ループ・横展開設計

位置づけ: 製造業BPO 16モジュールを起点に、データ取得方法・入力負荷・学習精度向上・6業種横展開の共通アーキテクチャを定義する
上位文書: c_02（プロダクト設計）、d_02（フィードバック学習ループ設計）、e_02b/e_02c（製造業BPO詳細/経営インテリジェンス設計）
対象: 製造業BPO 16モジュール（①-⑧ 現場BPO + ⑨-⑬ 経営インテリジェンス + ⑭-⑯ バックオフィス）
実装場所: `workers/bpo/manufacturing/` , `brain/genome/data/bpo/` , `shared/`

---

0. エグゼクティブサマリー

```
課題:
  ・16モジュールのデータ取得方法が設計書ごとに散在。顧客の入力負荷が不透明
  ・学習ループの「いつ・どれだけのデータで・どの精度まで」が未定義
  ・6業種展開時の共通基盤と業種固有部分の境界が曖昧

本設計で定義すること:
  A. 16モジュール×3カテゴリのデータ取得方法マトリクス + 初回セットアップウィザード
  B. 学習ループの統計的有意性・匿名化・反映タイミング・精度測定・フィードバックUI
  C. Base/Industry分離パターン + ゲノムJSON化による横展開インターフェース

設計原則:
  ・データが50%でも50%の価値を出す（グレースフルデグラデーション）
  ・初回セットアップ60分以内。月次更新は自動+CSV10分
  ・学習は3フェーズ（c_02 §3.1準拠）: ルールベース→企業適応→N社ネットワーク
```

A. データ鮮度と入力負荷の設計

A.1 データ取得方法マトリクス（16モジュール×3カテゴリ）

凡例: `A` =API自動（手間ゼロ） / `C` =CSV半自動（月1回10分） / `M` =手動入力（初回のみ）

#	モジュール	API自動(A)	CSV半自動(C)	手動入力(M)	月次更新工数
1	見積AI	-	-	加工種別マスタ, チャージレート, 材料単価マスタ	0分（マスタ更新時のみ）
2	生産計画AI	-	受注データCSV, 設備稼働実績CSV	工程マスタ, 設備登録	10分（受注CSV取込）
3	品質管理	-	検査記録CSV, 測定データCSV	管理項目設定, 規格値設定	10分（検査CSV取込）
4	在庫最適化	-	在庫棚卸CSV, 入出庫実績CSV	ABC分類設定, 安全在庫日数	10分（棚卸CSV取込）
5	SOP管理	-	-	手順書テキスト/写真	0分（更新時のみ）
6	設備保全	-	保全記録CSV	設備登録, 保全スケジュール	5分（保全CSV取込）
7	仕入管理	freee仕入データ	銀行明細CSV	仕入先マスタ, BOM登録	5分（銀行CSV取込）
8	ISO文書管理	-	-	ISO文書登録	0分（改訂時のみ）
9	PLインテリジェンス	freee月次PL	銀行明細CSV	加工種別マスタ（①と共通）	5分（freee自動+確認）
10	設備投資シミュレーター	-	-	検討設備仕様（都度入力）	0分（シミュレーション都度）
11	立地戦略AI	経産省/JETRO API	-	希望条件（都度入力）	0分（分析都度）
12	市場インテリジェンス	為替API, LME API, 経産省	-	主力材質・取引通貨設定	0分（全自動）
13	技術戦略AI	-	-	保有技術棚卸（⑤と連動）	0分（⑤更新時自動連携）
14	給与計算(Phase2+)	freee給与	-	従業員マスタ	0分（全自動）
15	勤怠管理(Phase2+)	kintone/LINE WORKS	勤怠CSV	勤務ルール設定	5分（確認のみ）
16	届出管理(Phase2+)	-	-	届出テンプレート選択	0分（発生時のみ）

月次合計工数: API自動モジュール=0分、CSV取込=約50分/月、手動=初回のみ

A.2 データ取得カテゴリ別UX設計

A.2.1 API自動（手間ゼロ）

連携先	対象モジュール	認証方式	データ取得頻度	Phase
freee	⑦⑨⑭	OAuth 2.0	日次（PLは月次）	Phase 1
為替API（ECB/日銀）	⑫	APIキー	日次	Phase 1
LME API（金属相場）	⑫	APIキー	日次	Phase 1
経産省 e-Stat	⑪⑫	公開API	月次	Phase 1
kintone	②⑮	APIトークン	リアルタイム	Phase 2
LINE WORKS	⑮	OAuth 2.0	日次	Phase 2

認証フロー設計:

```
freee OAuth:
  1. 「freee連携」ボタン → freee認証画面（リダイレクト）
  2. ユーザーが承認 → callback でアクセストークン取得
  3. リフレッシュトークンをSupabase暗号化カラムに保存
  4. 日次バッチでトークンリフレッシュ + データ取得
  → Phase 1: 手動OAuth。Phase 2: 自動リフレッシュ

為替/LME API:
  1. 初回設定で主力通貨・材質を選択（USD/EUR, SUS304/A5052等）
  2. 日次CRONで自動取得 → market_data テーブルに蓄積
  3. 閾値超過（前日比±3%）でSlack自動アラート
```

データ取得失敗時のフォールバック:

```
APIエラー → 3回リトライ（exponential backoff: 1s, 5s, 30s）
  → 失敗 → 前日のデータを「推定値」として表示（バッジ: ⚠ 推定値）
  → 24時間以上の取得失敗 → Slack通知 + ダッシュボードに警告バナー
  → CSV手動アップロードへのフォールバック導線を表示
```

A.2.2 CSV半自動（月1回10分）

対応フォーマット:

```
受注データCSV:    受注番号, 製品名, 数量, 納期, 工程1, 工程2, ...
生産実績CSV:      日付, 設備名, 受注番号, 稼働時間, 良品数, 不良数
検査記録CSV:      日付, 製品名, 検査項目, 測定値, 規格上限, 規格下限
在庫棚卸CSV:      品目コード, 品名, 現在庫数, 単価, カテゴリ
銀行明細CSV:      取引日, 摘要, 入金, 出金, 残高
保全記録CSV:      日付, 設備名, 保全種別(定期/突発), 内容, 費用, 停止時間
勤怠CSV:          従業員名, 日付, 出勤時刻, 退勤時刻, 休憩時間
```

CSV取込UX設計:

```
1. ドラッグ&ドロップ or ファイル選択
2. プレビュー画面:
   ・先頭5行を表示
   ・カラム自動マッピング（ヘッダー名のファジーマッチング）
   ・マッピング修正UI（ドロップダウンで正しいカラムを選択）
3. バリデーション結果:
   ・✅ 正常: 128行
   ・⚠ 警告: 3行（数値フォーマット不正 → 自動変換済み）
   ・❌ エラー: 1行（必須項目欠落 → スキップ）
4. 「取込実行」ボタン → 処理完了通知

設計参考: freee のCSVインポート機能
  ・ヘッダー行の自動検出
  ・文字コード自動判定（Shift_JIS/UTF-8）
  ・重複データの検出（取引日+摘要+金額の一致）
```

カラム自動マッピングルール:

```
COLUMN_MAPPING_RULES = {
    "受注データ": {
        "受注番号|注文番号|PO|order_no": "order_number",
        "製品名|品名|product|item": "product_name",
        "数量|個数|qty|quantity": "quantity",
        "納期|納入日|delivery|due_date": "delivery_date",
    },
    "銀行明細": {
        "取引日|日付|date": "transaction_date",
        "摘要|内容|description": "description",
        "入金|収入|credit": "credit_amount",
        "出金|支出|debit": "debit_amount",
    },
}
# ファジーマッチング: Levenshtein距離 ≤ 2 or 部分一致で候補提示
```

月次リマインダー設計:

```
毎月1日 09:00 Slack通知:
  「先月の生産実績CSVと在庫棚卸CSVのアップロードをお願いします」
  → アップロード画面へのディープリンク付き
  → 未アップロード時: 3日 09:00 に再通知
  → 7日経過: ダッシュボードに「データ未更新」バッジ表示

通知チャネル優先度: Slack > LINE WORKS > メール
通知抑制: 該当月のCSVが既にアップロード済みなら通知しない
```

A.2.3 手動入力（初回のみ）

データ種別	入力UI	テンプレート有無	所要時間	更新頻度
加工種別マスタ	テンプレート選択+カスタマイズ	金属/樹脂/板金/食品4種	15分	年1-2回
チャージレート	加工種別に紐づけて入力	業界相場プリセット	10分	半年1回
材料単価マスタ	LME連動+手動オーバーライド	JIS材質50種プリセット	5分	自動更新
設備登録	フォーム（設備名/種別/年式/価額）	-	30分（10台）	購入時
工程マスタ	設備に紐づけて自動生成	設備種別から自動推定	5分（確認のみ）	設備変更時
技能マトリクス	従業員×工程のスキルレベル	3段階（初心者/一人前/熟練）	20分（20名）	半年1回
顧客マスタ	上位10社+売上比率	-	10分	年1回
管理項目設定	品質規格値の設定	JIS規格プリセット	15分	製品追加時
安全在庫日数	品目カテゴリ別に設定	ABC分析推奨値プリセット	5分	半年1回
仕入先マスタ	主要仕入先+品目	-	10分	仕入先変更時
BOM登録	製品→部品展開	-	30分（主要10製品）	新製品時
保全スケジュール	設備×保全周期	メーカー推奨値プリセット	10分	年1回
ISO文書登録	ファイルアップロード+メタ情報	ISO9001条項テンプレート	60分	改訂時

A.3 初回セットアップウィザード

```
┌──────────────────────────────────────────────────────────┐
│  シャチョツー 初回セットアップ（製造業）                    │
│  全5ステップ / 約60分 / どのステップもスキップ可能          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Step 1: freee連携（3分）                                 │
│  ┌──────────────────────────────────────────────────┐    │
│  │ [freee連携する] ← OAuth認証ボタン                  │  │
│  │ [スキップ] ← CSVで代替可能                          │  │
│  │                                                    │  │
│  │ 取得データ: PL, 仕入データ, 勘定科目               │  │
│  │ スキップ時の影響:                                  │  │
│  │   ⑦仕入管理, ⑨PLインテリジェンスが手動CSV依存     │  │
│  │   → 月次10分の追加工数                             │  │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  Step 2: 業種テンプレート選択 + 加工種別設定（15分）      │
│  ┌──────────────────────────────────────────────────┐    │
│  │ テンプレートギャラリー:                            │  │
│  │                                                    │  │
│  │ [金属加工] 旋盤/MC/研磨/ワイヤカット/表面処理      │  │
│  │  └ サブ: 精密加工/自動車部品/航空機部品/一般機械   │  │
│  │                                                    │  │
│  │ [樹脂成型] 射出/押出/ブロー/真空                   │  │
│  │  └ サブ: 家電/自動車/医療/食品容器                 │  │
│  │                                                    │  │
│  │ [板金加工] レーザー/プレス/溶接/塗装               │  │
│  │  └ サブ: 精密板金/建築金物/制御盤/配電盤           │  │
│  │                                                    │  │
│  │ [食品OEM] 調合/充填/包装/検査                      │  │
│  │  └ サブ: 飲料/菓子/調味料/冷凍食品                 │  │
│  │                                                    │  │
│  │ テンプレート選択後:                                │  │
│  │   ・加工種別リストが自動設定される                 │  │
│  │   ・チャージレートに業界相場がプリセットされる     │  │
│  │   ・材料単価にJIS材質がプリセットされる            │  │
│  │   → 必要に応じてカスタマイズ（追加/削除/単価修正） │  │
│  │                                                    │  │
│  │ スキップ時の影響:                                  │  │
│  │   ①見積AI, ⑨PLインテリジェンスの精度が低下        │  │
│  │   → 後から設定画面で追加可能                       │  │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  Step 3: 設備登録（30分 / 1台3分×10台想定）              │
│  ┌──────────────────────────────────────────────────┐    │
│  │ 入力フォーム（1台あたり）:                         │  │
│  │   ・設備名（例: CNC旋盤 #1）                       │  │
│  │   ・種別（Step 2のテンプレートからドロップダウン） │  │
│  │   ・メーカー/型式（任意）                          │  │
```

```
│  │   ・取得年                                          │  │
│  │   ・取得価額（円）                                  │  │
│  │   ・消費電力（kW）← 種別から自動推定、手動上書き可 │  │
│  │                                                     │  │
│  │ 一括登録: CSVアップロード対応                       │  │
│  │ スキップ時の影響:                                   │  │
│  │   ②生産計画, ⑥設備保全, ⑩設備投資が利用不可      │  │
│  │   → ①見積AI, ⑨PLインテリジェンスは利用可能        │  │
│  └─────────────────────────────────────────────────┘  │
│                                                         │
│  Step 4: 主要顧客登録（10分）                           │
│  ┌─────────────────────────────────────────────────┐  │
│  │ 入力: 上位10社の社名 + 年間売上比率(%)            │  │
│  │                                                     │  │
│  │ 例:                                                 │  │
│  │  A社（自動車部品）  35%                             │  │
│  │  B社（産業機器）    20%                             │  │
│  │  C社（医療機器）    15%                             │  │
│  │  その他              30%                            │  │
│  │                                                     │  │
│  │ 自動算出: 顧客集中度（HHI指数）                     │  │
│  │   → HHI > 2500: 「特定顧客への依存度が高いです」   │  │
│  │                                                     │  │
│  │ スキップ時の影響:                                   │  │
│  │   ⑫市場インテリジェンスの「自社影響度」が低精度    │  │
│  │   → 後から設定画面で追加可能                        │  │
│  └─────────────────────────────────────────────────┘  │
│                                                         │
│  Step 5: 初回分析実行（3分 / 自動）                     │
│  ┌─────────────────────────────────────────────────┐  │
│  │ freee連携済みの場合:                                │  │
│  │   ・直近3ヶ月のPLデータを自動取得                   │  │
│  │   ・⑨PLインテリジェンスの初回分析を自動実行         │  │
│  │   ・業界ベンチマークとの比較結果を表示              │  │
│  │                                                     │  │
│  │ freee未連携の場合:                                  │  │
│  │   ・「PLデータCSVをアップロードしてください」       │  │
│  │   ・または「後でセットアップを続ける」ボタン        │  │
│  │                                                     │  │
│  │ 完了画面:                                           │  │
│  │   ・「初回セットアップ完了! 利用可能モジュール:」  │  │
│  │   ・ ✅ 見積AI（加工種別マスタ設定済み）            │  │
│  │   ・ ✅ PLインテリジェンス（PL取得済み）            │  │
│  │   ・ ⬜ 生産計画AI（受注CSVアップロードで有効化）   │  │
│  │   ・ ⬜ 品質管理（検査記録CSVで有効化）             │  │
│  └─────────────────────────────────────────────────┘  │
│                                                         │
│  合計: 約60分で①⑨が即利用可能                           │
│  残りモジュールは月次CSVアップロードで段階的に有効化    │
└─────────────────────────────────────────────────────────┘
```

セットアップ進捗の永続化:

```
# DB: company_setup_progress テーブル
{
    "company_id": "uuid",
    "industry": "manufacturing",
    "template": "metal_machining",
    "steps_completed": ["freee_connect", "template_select"],
    "steps_skipped": ["equipment_register"],
    "modules_available": ["quoting", "pl_intelligence"],
    "modules_pending": ["production_planning", "quality_control"],
    "setup_started_at": "2026-04-01T10:00:00Z",
    "setup_completed_at": null,
    "total_setup_minutes": 45
}
```

A.4 グレースフルデグラデーション設計

> **原則**: データが50%でも50%の価値を出す。欠損データのあるモジュールは「低精度モード」で稼働させ、利用不可にはしない。

モジュール別デグラデーション定義

#	モジュール	必須データ	あれば精度向上	データ欠損時の挙動
1	見積AI	加工種別(1つ以上)	過去見積実績, 材料単価マスタ	テンプレート相場で見積。confidence=0.5表示
2	生産計画AI	受注データ(1件以上), 設備(1台以上)	稼働実績, 工程マスタ	デフォルトキャパ(8h/日)で計算。精度バッジ「参考値」
3	品質管理	検査記録(10件以上)	SPC用30件以上	10件未満: 個別値一覧のみ。管理図・Cpk算出不可
4	在庫最適化	品目リスト	入出庫履歴3ヶ月	ABC分析のみ。EOQ/安全在庫は「仮値」表示
5	SOP管理	手順テキスト(1件以上)	写真/動画	テキストのみのSOP生成。多言語翻訳は可能
6	設備保全	設備登録(1台以上)	保全履歴6ヶ月	メーカー推奨周期でカレンダー生成。MTBF算出不可
7	仕入管理	仕入先(1社以上)	freee連携, BOM	BOM未登録: MRP不可。手動発注書のみ
8	ISO文書管理	ISO文書(1件以上)	全規格文書	アップロード済み文書のみ管理。欠損条項を一覧表示
9	PLインテリジェンス	PLデータ(1ヶ月)	3ヶ月以上, 設備データ	1ヶ月: 全社PLのみ。按分不可→「加工種別PLには3ヶ月必要」
10	設備投資シミュレーター	検討設備仕様	現有設備の稼働データ	業界相場でシミュレーション。「精度は±30%」と明記
11	立地戦略AI	希望条件	現有拠点情報	公開データのみでスコアリング。精度表示つき
12	市場インテリジェンス	主力材質設定	顧客マスタ, 輸出比率	為替/相場レポートのみ。自社インパクト分析は不可
13	技術戦略AI	保有技術(1件以上)	品質データ, 受注実績	定性的ロードマップのみ。定量評価は「データ不足」表示

デグラデーションUI設計

```
■ 精度バッジ（全モジュール共通）
  🟢 高精度（データ充足率 80%以上）
  🟡 標準精度（データ充足率 50–80%）
  🟠 参考値（データ充足率 30–50%）
  🔴 データ不足（30%未満 → 利用不可ではなく「限定機能」モード）

■ データ充足プログレスバー

  ┌──────────────────────────────────────┐
  │ 見積AI のデータ充足率: ▓▓▓
```

B. 学習ループの精度向上メカニズム

B.1 ベンチマークの統計的有意性

> **前提**: c_02 §3.1の3フェーズパターン（ルールベース→企業適応→N社ネットワーク）に準拠

B.1.1 必要サンプル数の定義

閾値	社数(N)	ラベル	表示	統計的根拠
N < 5	1-4社	非公開	表示しない	k-匿名性未達。個社特定リスク
5 ≤ N < 10	5-9社	傾向値	「同業5社の傾向」	記述統計のみ。中央値を表示
10 ≤ N < 30	10-29社	参考値	「同業10社以上の参考値」	平均値+標準偏差を表示。信頼区間は広い
30 ≤ N < 100	30-99社	ベンチマーク	「業界ベンチマーク（N=30+）」	中心極限定理で正規分布近似可能。95%信頼区間表示
N ≥ 100	100社以上	高信頼	「業界標準値（N=100+）」	セグメント別分析可能。パーセンタイル表示

参考比較:
- TKC BAST: 約27万社（全業種）。製造業だけでも数万社
- 中小企業庁 経営指標: 約2.5万社
- シャチョツーのターゲット: 製造業38万社のうちT2/T3=約4.6万社

B.1.2 セグメント別の必要サンプル数

```
■ セグメンテーション軸
  Axis 1: 加工種別（金属切削/板金/樹脂成型/食品OEM/表面処理/その他）
  Axis 2: 従業員規模（10-30名/31-100名/101-300名）
  Axis 3: 主力顧客業界（自動車/電子機器/産業機器/医療/航空/一般機械）

■ セグメント数 = 6 × 3 × 6 = 108セグメント
  → 全セグメントN=30は 108 × 30 = 3,240社必要
  → 非現実的。段階的にセグメント粒度を粗くする

■ 段階的セグメンテーション戦略
  Phase 1（~100社）: 加工種別のみ（6セグメント）
    → 各セグメントN=15-20で「参考値」として公開
    → 例: 「金属切削の粗利率: 20-30%（N=18社）」

  Phase 2（100-500社）: 加工種別 × 規模（18セグメント）
    → 主要セグメントでN=30達成
    → 例: 「金属切削・30-100名の粗利率: 22-28%（N=35社）」

  Phase 3（500社~）: 3軸フルセグメント
    → 主要セグメントでN=100達成
    → 例: 「金属切削・30-100名・自動車部品の粗利率: 25±3%（N=120社）」

■ パイロット期（~30社）の対応
  → セグメントなし。製造業全体の平均値のみ
  → 「製造業全体の粗利率: 18-32%（N=25社）」
  → 初期値は経産省/TKC BASTの公開統計値をフォールバック
```

B.1.3 ベンチマーク統計処理

```
def calculate_benchmark(segment_data: list[dict], metric: str) -> BenchmarkResult:
    """
    セグメント別ベンチマーク算出
    ロバスト統計: 外れ値の影響を最小化
    """
    values = [d[metric] for d in segment_data]
    n = len(values)

    if n < 5:
        return BenchmarkResult(status="HIDDEN", reason="k-anonymity violation")

    # ロバスト統計（外れ値耐性）
    median = np.median(values)
    q1, q3 = np.percentile(values, [25, 75])
    iqr = q3 - q1

    # Tukey法で外れ値除外
    lower_fence = q1 - 1.5 * iqr
    upper_fence = q3 + 1.5 * iqr
    filtered = [v for v in values if lower_fence <= v <= upper_fence]

    # 信頼区間（N≥30の場合のみ）
    if n >= 30:
        mean = np.mean(filtered)
        se = np.std(filtered, ddof=1) / np.sqrt(len(filtered))
        ci_95 = (mean - 1.96 * se, mean + 1.96 * se)
    else:
        mean = np.mean(filtered)
        ci_95 = None  # サンプル不足

    return BenchmarkResult(
        n=n,
        mean=round(mean, 2),
        median=round(median, 2),
        q1=round(q1, 2),
        q3=round(q3, 2),
        ci_95=ci_95,
        status="BENCHMARK" if n >= 30 else "REFERENCE" if n >= 10 else "TREND",
    )
```

B.2 匿名化ルール

> **法令参照**: 個人情報保護法第2条（個人情報の定義）、第43条（匿名加工情報の取扱い）

B.2.1 k-匿名性の実装

```
■ 基本ルール: k ≥ 5
  同一セグメント内に5社以上存在しない限り、ベンチマーク値を公開しない

■ セグメント判定
  セグメント = (加工種別, 従業員規模, 地域ブロック)
  例: (金属切削, 30-100名, 関東) に5社以上いればベンチマーク表示可

■ l-多様性の追加制約: l ≥ 3
  同一セグメント内の数値分布に3社以上の異なる値域が含まれること
  → 1社の突出した値で推定されることを防止
  例: 粗利率が全5社とも28%前後 → 特定社の粗利率が推測可能 → 公開しない

■ t-近似性 (Phase 2+)
  セグメント内の値分布が全体分布からt以上乖離しないことを検証
```

B.2.2 匿名化処理パイプライン

```
class AnonymizationPipeline:
    """匿名化処理パイプライン（d_02 §3 フィードバック学習ループと連動）"""

    def anonymize_for_benchmark(self, company_data: dict) -> dict | None:
        """個社データをベンチマーク集約用に匿名化"""

        # Step 1: 直接識別子の除去
        anonymized = {
            k: v for k, v in company_data.items()
            if k not in DIRECT_IDENTIFIERS
        }

        # Step 2: 準識別子の一般化
        anonymized["employee_count"] = generalize_range(
            company_data["employee_count"],
            ranges=[(10, 30), (31, 100), (101, 300)]
        )
        anonymized["region"] = generalize_to_block(
            company_data["prefecture"]  # 都道府県→8地域ブロック
        )
        anonymized["annual_revenue"] = generalize_range(
            company_data["annual_revenue"],
            ranges=[(0, 1e8), (1e8, 5e8), (5e8, 10e8), (10e8, 50e8)]
        )

        # Step 3: k-匿名性チェック
        segment = self.get_segment(anonymized)
        segment_count = self.count_segment(segment)
        if segment_count < K_THRESHOLD:  # K_THRESHOLD = 5
            return None  # ベンチマーク非公開

        # Step 4: l-多様性チェック
        diversity = self.check_l_diversity(segment, metric="gross_margin")
        if diversity < L_THRESHOLD:  # L_THRESHOLD = 3
            return None

        # Step 5: ノイズ付加（差分プライバシー）
        # 具体的な数値にラプラスノイズを加算
        for metric in NUMERIC_METRICS:
            epsilon = 0.1  # プライバシーバジェット
            sensitivity = METRIC_SENSITIVITY[metric]
            noise = np.random.laplace(0, sensitivity / epsilon)
            anonymized[metric] += noise

        return anonymized

# 直接識別子: 絶対に集約データに含めない
DIRECT_IDENTIFIERS = {
    "company_id", "company_name", "representative_name",
```

```
    "address", "phone", "email", "bank_account",
    "customer_names", "supplier_names",
}

# 準識別子の一般化（都道府県→地域ブロック）
REGION_BLOCKS = {
    "北海道": "北海道",
    "青森県": "東北", "岩手県": "東北", "宮城県": "東北",
    "秋田県": "東北", "山形県": "東北", "福島県": "東北",
    "茨城県": "関東", "栃木県": "関東", "群馬県": "関東",
    "埼玉県": "関東", "千葉県": "関東", "東京都": "関東", "神奈川県": "関東",
    # ... 以下省略
}
```

B.2.3 同意取得設計

```
■ 同意フロー（初回セットアップ Step 0 で取得）
  「匿名化されたデータを業界ベンチマークに活用してよいですか？」

  [同意する] → ベンチマーク参加 + ベンチマーク閲覧可能
  [同意しない] → ベンチマーク不参加 + ベンチマーク閲覧不可

  補足説明:
  ・「御社のデータは匿名化され、個社特定は不可能です」
  ・「同一セグメントに5社以上いない場合、データは集約対象外です」
  ・「いつでも同意撤回可能です」

■ consent_records テーブルに記録（セキュリティ設計a_01準拠）
  consent_type = "benchmark_participation"
  granted = true/false
  granted_at = timestamp
  ip_address, user_agent（同意証跡）
```

B.3 学習結果の反映タイミング

```
■ リアルタイム（秒単位）
  ・設備停止アラート → ⑥設備保全のMTBF閾値超過検知
  ・品質逸脱アラート → ③品質管理の管理図上下限逸脱
  ・為替急変アラート → ⑫市場インテリジェンスの閾値（前日比±3%）
  ・在庫切れ予測 → ④在庫最適化の安全在庫割れ

  実装: WebSocket or Server-Sent Events（SSE）
  → Phase 1はポーリング（30秒間隔）で代替

■ 日次（毎日 06:00 JST バッチ）
  ・ヘルススコア更新 → brain/twin のcompany_state_snapshots更新
  ・在庫アラート一覧 → ④在庫最適化の翌日発注リスト
  ・為替/相場データ取得 → ⑫市場インテリジェンス
  ・設備稼働率サマリー → ②生産計画AIの翌日計画調整

■ 週次（毎週月曜 08:00 JST）
  ・受注トレンド分析 → ①見積AIの受注確率モデル更新
  ・稼働率週報 → ②生産計画AIのボトルネック分析
  ・品質週報 → ③品質管理のCpkトレンド
  ・保全作業完了チェック → ⑥設備保全の未実施アラート

■ 月次（毎月1日 03:00 JST）
  ・PL分析実行 → ⑨PLインテリジェンス
  ・ベンチマーク更新 → 全モジュールのベンチマーク値再計算
  ・市場レポート生成 → ⑫市場インテリジェンス月報
  ・学習モデル再トレーニング → 見積精度の再評価
  ・フィードバック集計 → 全モジュールの精度KPI更新

■ 四半期（1月/4月/7月/10月の1日）
  ・技術ロードマップ更新 → ⑬技術戦略AI
  ・投資判断レビュー → ⑩設備投資シミュレーターの予実対比
  ・ベンチマーク定義見直し → セグメント粒度の調整
  ・補助金DB更新 → ⑩設備投資シミュレーターの補助金情報
```

反映タイミング管理テーブル:

```
CREATE TABLE learning_schedule (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    module_name TEXT NOT NULL,          -- 'pl_intelligence', 'quoting' 等
    frequency TEXT NOT NULL,            -- 'realtime', 'daily', 'weekly', 'monthly',
'quarterly'
    cron_expression TEXT,               -- '0 3 1 * *' 等
    last_executed_at TIMESTAMPTZ,
    next_scheduled_at TIMESTAMPTZ,
    execution_status TEXT DEFAULT 'pending',  -- 'pending', 'running', 'completed',
'failed'
    execution_duration_ms INTEGER,
    company_id UUID REFERENCES companies(id),
    created_at TIMESTAMPTZ DEFAULT NOW()
);
-- RLS: company_id ベースのテナント分離
```

B.4 精度の測定方法

B.4.1 モジュール別精度KPI

#	モジュール	KPI	計算式	Phase 1目標	Phase 2目標	Phase 3目標
1	見積AI	見積精度(MAPE)	`mean(abs(AI見積 - 実績) / 実績)`	±20%	±10%	±5%
2	生産計画AI	納期遵守率	`納期内完了件数 / 全件数`	80%	90%	95%
3	品質管理	不良予測精度	`予測不良発生 / 実際不良発生` の適合率	60%	80%	90%
4	在庫最適化	欠品率	`欠品発生日数 / 営業日数`	5%	2%	0.5%
5	SOP管理	手順書採用率	`承認済み件数 / 生成件数`	50%	70%	85%
6	設備保全	故障予測精度	`予測通り故障 / 予測発報数`	40%	60%	80%
7	仕入管理	発注精度	`自動発注採用率`	30%	60%	80%
8	ISO文書管理	文書完備率	`AI生成文書の審査通過率`	60%	80%	95%
9	PLインテリジェンス	PL予測MAPE	`mean(abs(予測PL - 実績PL) / 実績PL)`	±20%	±10%	±5%
10	設備投資シミュレーター	回収予測乖離	`abs(予測回収年 - 実績回収年)`	±2年	±1年	±0.5年
11	立地戦略AI	推奨適合率	`推奨立地で満足した顧客率`	50%	70%	85%
12	市場インテリジェンス	需要予測相関	`corr(予測需要, 実績需要)`	0.5	0.7	0.85
13	技術戦略AI	ロードマップ採用率	`提案技術の実際投資率`	30%	50%	70%

B.4.2 精度測定の実装

```
class AccuracyTracker:
    """精度測定・記録（d_02 フィードバック学習ループと連携）"""

    async def record_prediction_vs_actual(
        self,
        company_id: str,
        module: str,
        prediction: dict,
        actual: dict,
        metric: str,
    ) -> AccuracyRecord:
        """予測値と実績値の乖離を記録"""

        predicted_value = prediction[metric]
        actual_value = actual[metric]

        # MAPE計算
        if actual_value != 0:
            mape = abs(predicted_value - actual_value) / abs(actual_value)
        else:
            mape = None

        record = AccuracyRecord(
            company_id=company_id,
            module=module,
            metric=metric,
            predicted_value=predicted_value,
            actual_value=actual_value,
            mape=mape,
            recorded_at=datetime.utcnow(),
        )

        # DB保存（既存 accuracy_logs テーブル）
        await self.save(record)

        # 月次集計の自動トリガー
        if await self.should_recalculate(company_id, module):
            await self.recalculate_monthly_accuracy(company_id, module)

        return record

    async def get_module_accuracy_summary(
        self, company_id: str, module: str, months: int = 3
    ) -> AccuracySummary:
        """直近N月の精度サマリー"""
        records = await self.get_records(company_id, module, months)

        return AccuracySummary(
            module=module,
```

```
            n_predictions=len(records),
            mean_mape=np.mean([r.mape for r in records if r.mape]),
            median_mape=np.median([r.mape for r in records if r.mape]),
            trend="improving" if self.is_improving(records) else "stable",
            phase=self.determine_phase(records),  # Phase 1/2/3
        )
```

B.4.3 精度ダッシュボード

```
┌──────────────────────────────────────────────────────────┐
│  精度ダッシュボード（製造業BPO）                              │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  全体精度スコア: 72/100（Phase 1基準で「良好」）            │
│  データ蓄積期間: 3ヶ月                                     │
│  次のフェーズアップ条件: 見積実績50件 → 現在32件            │
│                                                          │
│  モジュール別:                                             │
│  ① 見積AI        ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ MAPE 18%（目標±20%）✅  │
│  ② 生産計画      ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 納期遵守 75%（目標80%）⚠  │
│  ③ 品質管理      ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 予測精度 65%（目標60%）✅  │
│  ⑨ PLインテリ    ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ MAPE 15%（目標±20%）✅  │
│                                                          │
│  学習ループの状態:                                         │
│  Phase 1（ルールベース）: ①②③④⑤⑥⑦⑧⑨⑩⑪⑫⑬                 │
│  Phase 2（企業適応）: 移行準備中（データ蓄積3ヶ月到達）      │
│  Phase 3（N社ネットワーク）: 未到達（N=25社 / 目標50社）     │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

B.5 フィードバックUI

> **d_02 §2 モジュール別学習ループ設計の拡張版。**製造業16モジュール固有のフィードバック収集ポイントを定義。

B.5.1 フィードバック種別

```
■ Type 1: 承認/却下（バイナリ）
  適用: ①見積AI, ②生産計画AI, ⑤SOP管理, ⑦仕入管理

  UI:
    ┌──────────────────────────────────────┐
    │ AI見積: ¥177,000（confidence 0.82）     │
    │                                       │
    │  [✅ 承認]  [❌ 却下]  [✏️ 修正]        │
    └──────────────────────────────────────┘

  データ記録:
    feedback_type = "approval"
    action = "approved" | "rejected" | "modified"
    original_value = 177000
    modified_value = 185000  （修正時のみ）
    modification_reason = "表面処理の工数が過少"  （任意）

■ Type 2: 5段階評価（分析・レポート系）
  適用: ⑨PLインテリジェンス, ⑩設備投資シミュレーター, ⑪立地戦略AI, ⑫市場インテリジェンス, ⑬技術戦略
AI

  UI:
    ┌──────────────────────────────────────┐
    │ PLインテリジェンス 月次レポート          │
    │                                       │
    │ この分析は役に立ちましたか？             │
    │ ⭐⭐⭐⭐⭐                                 │
    │ 1:的外れ 2:微妙 3:普通 4:有用 5:的確    │
    │                                       │
    │ コメント（任意）:                       │
    │ [                                    ] │
    └──────────────────────────────────────┘

  データ記録:
    feedback_type = "rating"
    rating = 1-5
    comment = "射出成型の光熱費比率が実態と乖離"

■ Type 3: 実績入力（予測検証系）
  適用: ①見積AI（受注結果）, ②生産計画AI（実績稼働率）, ④在庫最適化（実績欠品）

  UI:
    ┌──────────────────────────────────────┐
    │ 見積 #A-2026-0142                      │
    │ AI見積: ¥177,000                       │
    │                                       │
    │ 結果を教えてください:                   │
    │ ○ 受注（実績金額: [        ] 円）       │
    │ ○ 失注（理由: [ドロップダウン]）         │
```

```
    │   → 価格負け / 納期負け / 品質要求 / 他 │
    │ ○ 保留中                                │
    └──────────────────────────────────────┘


  データ記録:
    feedback_type = "outcome"
    outcome = "won" | "lost" | "pending"
    actual_amount = 185000
    loss_reason = "price_competition"  (失注時)
```

B.5.2 フィードバックデータの学習反映フロー

```
                    フィードバック収集
                          │
                ┌─────────┼─────────┐
                ▼         ▼         ▼
            Type 1    Type 2    Type 3
            承認/却下  5段階評価  実績入力
                │         │         │
                ▼         ▼         ▼
            ┌──────────────────────────┐
            │ feedback_records テーブル保存 │
            │ (d_02 §2 既存テーブル拡張)   │
            └──────────────────────────┘
                          │
                          │
            ┌──────────────────────────┐
            │ 月次バッチ: 学習反映処理       │
            │                            │
            │ 1. フィードバック集計          │
            │    → 承認率, 修正パターン分析  │
            │                            │
            │ 2. パラメータ調整              │
            │    → チャージレート補正係数    │
            │    → ベンチマーク値の更新      │
            │    → 閾値の最適化              │
            │                            │
            │ 3. 精度KPI再計算               │
            │    → AccuracyTracker更新      │
            │                            │
            │ 4. Phase判定                  │
            │    → Phase 1→2の移行判定      │
            └──────────────────────────┘

■ Phase移行判定基準
  Phase 1 → Phase 2:
    ・フィードバック件数 ≥ 50件/モジュール
    ・利用期間 ≥ 3ヶ月
    ・精度KPIがPhase 1目標を達成

  Phase 2 → Phase 3:
    ・同業種テナント数 ≥ 50社
    ・ベンチマーク同意率 ≥ 80%
    ・k-匿名性(k≥5)を満たすセグメントが3以上
```

B.5.3 フィードバック促進設計

```
■ 収集タイミング（ユーザーの自然なワークフロー内で取得）
  ・見積送付時 → 「この見積を送信しますか？ [送信] [修正]」で承認/修正を取得
  ・見積結果判明時 → 週次で「未回答の見積結果があります」通知
  ・月次レポート閲覧後 → ページ下部に評価ウィジェット表示

■ フィードバック負荷の最小化
  ・必須: 承認/却下のみ（1クリック）
  ・任意: 修正理由、コメント、5段階評価
  ・自動: 実績データとの突合（freee連携で自動取得可能な場合）

■ フィードバック率の目標
  ・Type 1（承認/却下）: 90%以上（見積確定フローに組み込み）
  ・Type 2（5段階評価）: 30%以上（レポート閲覧者の3割）
  ・Type 3（実績入力）: 50%以上（受注/失注結果）
```

C. 横展開インターフェース設計

C.1 Base/Industry分離パターン

> **原則**: 共通ロジック（80%）をBaseクラスに、業種固有パラメータ（20%）をゲノムJSONに分離

C.1.1 パイプライン抽象化

```
# shared/base_pipeline.py

from abc import ABC, abstractmethod
from dataclasses import dataclass


@dataclass
class PipelineConfig:
    """ゲノムJSONから読み込まれるパイプライン設定"""
    industry: str                          # "manufacturing", "construction", ...
    cost_categories: list[str]             # 業種固有の原価区分
    allocation_method: str                 # 原価按分方法
    benchmarks: dict[str, tuple[float, float]]  # ベンチマーク値（min, max）
    thresholds: dict[str, float]           # 閾値
    templates: dict[str, str]              # レポートテンプレートID


class BasePLIntelligencePipeline(ABC):
    """PLインテリジェンスの共通基底クラス"""

    def __init__(self, config: PipelineConfig):
        self.config = config

    async def run(self, company_id: str, period: str) -> PLReport:
        """共通パイプラインフロー（Template Method）"""
        # Step 1: PLデータ取得（共通）
        raw_pl = await self.fetch_pl_data(company_id, period)

        # Step 2: コスト按分（業種固有）
        allocated = await self.allocate_costs(raw_pl)

        # Step 3: PL算出（共通エンジン）
        pl_results = self.calculate_pl(allocated)

        # Step 4: ベンチマーク比較（共通エンジン + 業種固有閾値）
        comparison = self.compare_benchmark(pl_results)

        # Step 5: レポート生成（共通エンジン + 業種固有テンプレート）
        report = await self.generate_report(pl_results, comparison)

        # Step 6: 異常検知（共通エンジン + 業種固有閾値）
        anomalies = self.detect_anomalies(pl_results)

        return PLReport(
            results=pl_results,
            comparison=comparison,
            anomalies=anomalies,
            report=report,
        )
```

```
    async def fetch_pl_data(self, company_id: str, period: str) -> dict:
        """PLデータ取得（共通: freee API / CSV）"""
        # 共通実装
        ...

    @abstractmethod
    async def allocate_costs(self, raw_pl: dict) -> dict:
        """コスト按分（業種固有: 製造業=設備稼働時間、建設業=工事別）"""
        ...

    def calculate_pl(self, allocated: dict) -> list[dict]:
        """PL算出（共通: 売上−原価=粗利の計算ロジック）"""
        results = []
        for category in self.config.cost_categories:
            revenue = allocated.get(f"{category}_revenue", 0)
            costs = sum(
                allocated.get(f"{category}_{cost_type}", 0)
                for cost_type in self.config.cost_categories
            )
            gross_profit = revenue - costs
            gross_margin = gross_profit / revenue * 100 if revenue else 0

            results.append({
                "category": category,
                "revenue": revenue,
                "total_cost": costs,
                "gross_profit": gross_profit,
                "gross_margin": round(gross_margin, 1),
            })
        return results

    def compare_benchmark(self, pl_results: list[dict]) -> list[dict]:
        """ベンチマーク比較（共通エンジン + ゲノムJSONの閾値）"""
        comparisons = []
        for result in pl_results:
            category = result["category"]
            benchmark = self.config.benchmarks.get(category, (0, 100))
            actual = result["gross_margin"]

            if actual < benchmark[0]:
                severity = "WARNING"
            elif actual > benchmark[1]:
                severity = "EXCELLENT"
            else:
                severity = "NORMAL"

            comparisons.append({
                "category": category,
                "actual": actual,
                "benchmark_low": benchmark[0],
                "benchmark_high": benchmark[1],
```

```
                "severity": severity,
            })
        return comparisons


# 製造業固有の実装
class ManufacturingPLIntelligencePipeline(BasePLIntelligencePipeline):
    """製造業PLインテリジェンス（e_02c §9準拠）"""

    async def allocate_costs(self, raw_pl: dict) -> dict:
        """設備稼働時間ベースの原価按分"""
        # INDIRECT_COST_ALLOCATION ルール（e_02c §9.3 Step 2準拠）
        # 光熱費 = 設備別消費電力kW × 稼働時間 × 電力単価
        # 間接工 = 直接工の比率で配賦
        # 管理部門 = 売上比率で按分
        ...


# 建設業固有の実装
class ConstructionPLIntelligencePipeline(BasePLIntelligencePipeline):
    """建設業PLインテリジェンス"""

    async def allocate_costs(self, raw_pl: dict) -> dict:
        """工事別原価按分"""
        # 材料費: 工事別に直課
        # 外注費: 工事別に直課
        # 重機費: 工事日報の使用時間で按分
        # 現場管理費: 工事規模比率で按分
        ...
```

C.1.2 業種別設定比較表

設計要素	製造業	建設業	医療福祉	不動産	物流	卸売
原価区分	材料/労務/光熱/減価償却/外注	材料/労務/外注/重機/仮設	人件費/医薬品/医療材料/設備	仲介手数料/広告/人件費	燃料/車両/人件費/倉庫	仕入/物流/人件費/倉庫
按分方法	設備稼働時間	工事別原価	診療科別	物件別	配送ルート別	商品カテゴリ別
KPIの軸	加工種別粗利率	工事粗利率	診療科別収益	物件種別利回り	配送効率	商品回転率
ベンチマーク源	経産省工業統計	国交省建設統計	厚労省病院経営	国交省不動産統計	国交省物流統計	経産省商業統計
季節変動	年度末集中	天候依存+年度末	インフルエンザ/花粉	引越しシーズン	歳暮/中元	歳暮/中元/年度末

C.2 ゲノムJSON化

> **実装場所**: `brain/genome/data/bpo/manufacturing_intelligence.json`
> **既存パターン**: `brain/genome/data/manufacturing.json` （見積AI用）を拡張

C.2.1 ゲノムJSON構造

```
// brain/genome/data/bpo/manufacturing_intelligence.json
{
  "industry": "manufacturing",
  "version": "1.0.0",
  "modules": {
    "pl_intelligence": {
      "cost_categories": [
        "material", "labor", "utility", "depreciation", "outsource"
      ],
      "allocation_method": "machine_hours",
      "direct_cost_mapping": {
        "材料費": "process_type_material_consumption",
        "外注費": "process_type_outsource",
        "工具消耗品費": "equipment_consumables"
      },
      "indirect_cost_allocation": {
        "労務費_直接工": "direct_labor_time_ratio",
        "労務費_間接工": "direct_labor_ratio_prorate",
        "管理部門人件費": "revenue_ratio",
        "光熱費": "equipment_power_time",
        "減価償却費": "equipment_book_value",
        "地代家賃": "floor_area_ratio"
      },
      "benchmarks": {
        "CNC旋盤":     {"gross_margin": [25, 35], "utility_ratio": [3, 5],
"labor_ratio": [30, 40]},
        "MCセンタ":    {"gross_margin": [20, 30], "utility_ratio": [4, 7],
"labor_ratio": [25, 35]},
        "板金":        {"gross_margin": [20, 28], "utility_ratio": [2, 4],
"labor_ratio": [35, 45]},
        "射出成型":    {"gross_margin": [15, 25], "utility_ratio": [8, 15], "labor_ratio":
[20, 30]},
        "表面処理":    {"gross_margin": [30, 45], "utility_ratio": [5, 10], "labor_ratio":
[25, 35]}
      },
      "anomaly_rules": {
        "month_over_month_threshold": 0.20,
        "consecutive_decline_months": 3,
        "margin_floor": 0.10
      },
      "report_template": "manufacturing_pl_monthly"
    },
    "capex_simulator": {
      "discount_rate": 0.05,
      "investment_criteria": {
        "GO":             {"payback_max": 3},
        "GO_CONDITIONAL": {"payback_max": 5},
        "WAIT":           {"payback_max": 7},
        "NO_GO":          {"payback_max": 999}
```

```
      },
      "equipment_price_db": {
        "CNC旋盤":  {"new": [15000000, 50000000], "used": [3000000, 15000000],
"useful_life": 10},
        "MCセンタ": {"new": [20000000, 80000000], "used": [5000000, 30000000],
"useful_life": 10}
      },
      "sensitivity_params": {
        "utilization_rate": 0.10,
        "charge_rate": 0.15,
        "material_cost": 0.20
      }
    },
    "market_intelligence": {
      "external_data_sources": [
        {"name": "ecb_exchange", "type": "api", "frequency": "daily"},
        {"name": "lme_metals", "type": "api", "frequency": "daily"},
        {"name": "meti_production_index", "type": "api", "frequency": "monthly"}
      ],
      "alert_thresholds": {
        "exchange_rate_daily_change": 0.03,
        "metal_price_daily_change": 0.05,
        "production_index_monthly_change": 0.10
      },
      "material_categories": [
        "steel", "stainless", "aluminum", "copper", "resin"
      ]
    }
  },
  "setup_wizard": {
    "templates": {
      "metal_machining": {
        "process_types": ["CNC旋盤", "MCセンタ", "研磨", "ワイヤーカット", "表面処理"],
        "default_charge_rates": {"CNC旋盤": 8000, "MCセンタ": 9000, "研磨": 10000}
      },
      "resin_molding": {
        "process_types": ["射出成型", "押出成型", "ブロー成型", "真空成型"],
        "default_charge_rates": {"射出成型": 8000, "押出成型": 6000}
      },
      "sheet_metal": {
        "process_types": ["レーザー加工", "プレス", "溶接", "塗装"],
        "default_charge_rates": {"レーザー加工": 8000, "プレス": 5000, "溶接": 7000}
      },
      "food_oem": {
        "process_types": ["調合", "充填", "包装", "検査"],
        "default_charge_rates": {"調合": 5000, "充填": 4000, "包装": 3000}
      }
    }
  },
  "learning": {
    "phase_transition": {
      "phase1_to_phase2": {
```

```
        "min_feedback_count": 50,
        "min_months": 3,
        "min_accuracy_achievement": true
      },
      "phase2_to_phase3": {
        "min_tenants_same_industry": 50,
        "min_benchmark_consent_rate": 0.80,
        "min_k_anonymous_segments": 3
      }
    },
    "benchmark_update_frequency": "monthly",
    "k_anonymity_threshold": 5,
    "l_diversity_threshold": 3
  }
}
```

C.2.2 新業種追加の手順

```
■ 新業種追加（例: 不動産業）の作業リスト

1. ゲノムJSON作成（30分）
   → brain/genome/data/bpo/realestate_intelligence.json
   → 原価区分、按分方法、ベンチマーク値、閾値を定義

2. Industry固有クラス作成（2時間）
   → workers/bpo/realestate/pipelines/ に
     ManufacturingPLIntelligencePipeline と同構造のクラスを作成
   → allocate_costs() のみ業種固有実装

3. セットアップウィザードのテンプレート追加（30分）
   → ゲノムJSON の setup_wizard.templates に追加

4. テスト作成（1時間）
   → tests/workers/bpo/realestate/ にテスト

合計: 約4時間で新業種のインテリジェンスモジュール追加可能
```

C.2.3 ゲノムローダーの拡張

```
# brain/genome/loader.py の拡張

class GenomeLoader:
    """ゲノムJSONローダー（既存の loader.py を拡張）"""

    INTELLIGENCE_GENOME_DIR = "brain/genome/data/bpo/"

    async def load_intelligence_config(
        self, industry: str, module: str
    ) -> PipelineConfig:
        """インテリジェンスモジュール用の設定をロード"""
        path = f"{self.INTELLIGENCE_GENOME_DIR}{industry}_intelligence.json"
        genome = await self._load_json(path)

        module_config = genome["modules"].get(module)
        if not module_config:
            raise ValueError(f"Module {module} not found for industry {industry}")

        return PipelineConfig(
            industry=industry,
            cost_categories=module_config.get("cost_categories", []),
            allocation_method=module_config.get("allocation_method", "revenue_ratio"),
            benchmarks=module_config.get("benchmarks", {}),
            thresholds=module_config.get("anomaly_rules", {}),
            templates=module_config.get("report_template", ""),
        )

    async def load_setup_templates(self, industry: str) -> dict:
        """セットアップウィザード用テンプレートをロード"""
        path = f"{self.INTELLIGENCE_GENOME_DIR}{industry}_intelligence.json"
        genome = await self._load_json(path)
        return genome.get("setup_wizard", {}).get("templates", {})
```

C.3 横展開ロードマップ

```
Phase 1（今）: 製造業で全16モジュール実装 + Base抽象化
  → 製造業固有の allocate_costs, benchmarks をJSON化
  → BasePipeline の共通ロジックを確定

Phase 2（PMF検証後）: 建設業に横展開
  → construction_intelligence.json 作成
  → ConstructionPLIntelligencePipeline（工事別原価）
  → 建設業固有: 工事進行基準、JV按分

Phase 3（子会社化後）: 残り4業種
  → 医療福祉: 診療科別収益、保険点数ベース
  → 不動産: 物件別利回り、仲介/管理の分離
  → 物流: ルート別コスト、車両稼働率
  → 卸売: 商品カテゴリ別、仕入条件最適化

各業種の追加コスト: ゲノムJSON（30分）+ 固有実装（2時間）+ テスト（1時間）= 約4時間
```

D. 実装優先度

優先度	対象	実装時期	工数
P0	初回セットアップウィザード（5ステップ）	Phase 1 Week 5	3日
P0	グレースフルデグラデーション（精度バッジ）	Phase 1 Week 5	1日
P0	CSV取込UI（カラム自動マッピング）	Phase 1 Week 5	2日
P1	フィードバックUI（Type 1: 承認/却下）	Phase 1 Week 6	1日
P1	AccuracyTracker（精度測定基盤）	Phase 1 Week 6	2日
P1	月次リマインダー（Slack通知）	Phase 1 Week 6	0.5日
P2	ベンチマーク統計エンジン	Phase 2 Week 1	3日
P2	匿名化パイプライン	Phase 2 Week 1	2日
P2	Base/Industry分離リファクタリング	Phase 2 Week 2	3日
P2	ゲノムJSON化（製造業）	Phase 2 Week 2	1日
P3	精度ダッシュボード	Phase 2 Week 3	2日
P3	フィードバックUI（Type 2,3）	Phase 2 Week 3	2日
P3	2業種目（建設業）横展開	Phase 2 Week 4	1週間

変更履歴

日付	版	変更内容
2026-03-28	1.0	初版作成